

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Тема: «Методы сортировки»

По дисциплине «Основы алгоритмизации и программирования»

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

1. Метод вставки

1. Постановка задачи

Отсортировать массив из n элементов по возрастанию.

2. Анализ задачи

1. Исходные данные: массив a размером n .

2. Массив делится на две части:

- отсортированная — элементы $a[0 \dots i-1]$;
- неотсортированная — элементы $a[i \dots n-1]$.

Отсортированная часть содержит один элемент $a[0]$, граница отделяет ее от неотсортированной части: $a[] = \{a[0] \mid a[1], \dots, a[n-1]\}$.

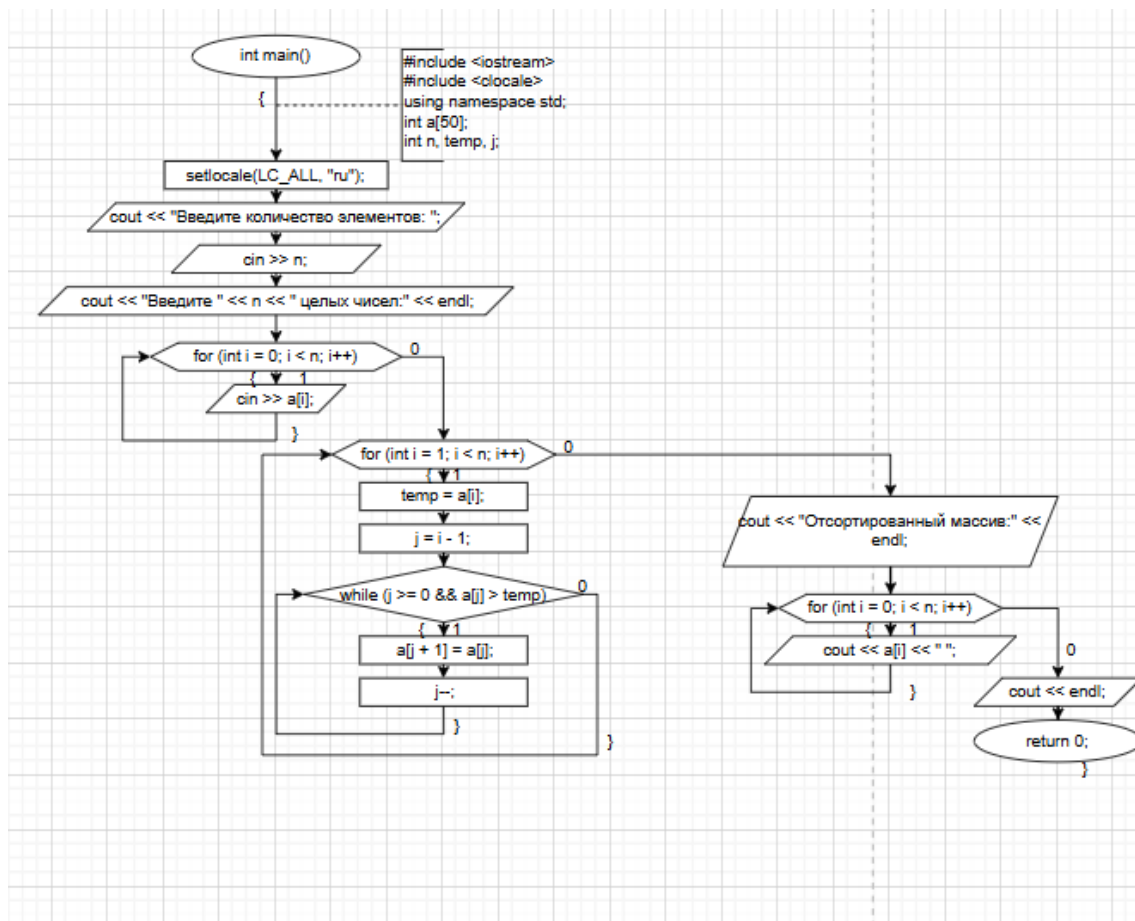
3. Внешний цикл `for (i = 1; i < n; i++)` перебирает элементы неотсортированной части. Сохраняем текущий элемент в переменную: $temp = a[i]$. Устанавливаем индекс $j = i - 1$ — последний элемент отсортированной части.

4. Внутренний цикл `while (j >= 0 && a[j] > temp)` ищет место для вставки неотсортированных элементов. Пока $a[j] > temp$, сдвигаем элемент вправо: $a[j + 1] = a[j]$, затем $j--$. Причина окончания цикла - найден элемент $a[j] \leq temp$ или достигнуто начало массива ($j = -1$).

5. Вставляем сохранённый элемент на освободившееся место: $a[j + 1] = temp$.

6. Граница отсортированной части автоматически расширяется на один элемент (переход к следующей итерации внешнего цикла).

3. Блок-схема



4. Код

```

#include <iostream>
#include <locale>
using namespace std;

int main() {
    setlocale(LC_ALL, "ru");
    int a[50];
    int n;

    cout << "Введите количество элементов: ";
    cin >> n;

    cout << "Введите " << n << " целых чисел:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    cout << "\nНачальный массив: ";
    for (int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }
    cout << "\n\n";

    for (int i = 1; i < n; i++) {
        int temp = a[i];
        int j = i - 1;

        cout << "Шаг " << i << ": вставляем элемент " << temp << endl;
        cout << "Текущий массив: ";
        for (int k = 0; k < n; k++) {

```

```

        if (k == i) {
            cout << "[" << a[k] << "]" ";
        }
        else {
            cout << a[k] << " ";
        }
    }
    cout << endl;

    while (j >= 0 && a[j] > temp) {
        a[j + 1] = a[j];

        cout << "  Сдвигаем " << a[j] << "  вправо: ";
        for (int k = 0; k < n; k++) {
            if (k == j + 1) {
                cout << "[" << a[k] << "]" ";
            }
            else if (k == i && k != j + 1) {
                cout << "[_]" ";
            }
            else {
                cout << a[k] << " ";
            }
        }
        cout << endl;

        j--;
    }

    a[j + 1] = temp;
    cout << "  Вставляем " << temp << "  на позицию " << j + 1 << ": ";
    for (int k = 0; k < n; k++) {
        if (k == j + 1) {
            cout << "[" << a[k] << "]" ";
        }
        else {
            cout << a[k] << " ";
        }
    }
    cout << "\n" << endl;
}

cout << "Отсортированный массив: ";
for (int i = 0; i < n; i++) {
    cout << a[i] << " ";
}
cout << endl;

return 0;
}

```

5. Результат выполнения программы

```

Введите количество элементов: 5
Введите 5 целых чисел:
50
40
5
9
3

Начальный массив: 50 40 5 9 3

Шаг 1: вставляем элемент 40
Текущий массив: 50 [40] 5 9 3
Сдвигаем 50 вправо: 50 [50] 5 9 3
Вставляем 40 на позицию 0: [40] 50 5 9 3

Шаг 2: вставляем элемент 5
Текущий массив: 40 50 [5] 9 3
Сдвигаем 50 вправо: 40 50 [50] 9 3
Сдвигаем 40 вправо: 40 [40] [ ] 9 3
Вставляем 5 на позицию 0: [5] 40 50 9 3

Шаг 3: вставляем элемент 9
Текущий массив: 5 40 50 [9] 3
Сдвигаем 50 вправо: 5 40 50 [50] 3
Сдвигаем 40 вправо: 5 40 [40] [ ] 3
Вставляем 9 на позицию 1: 5 [9] 40 50 3

Шаг 4: вставляем элемент 3
Текущий массив: 5 9 40 50 [3]
Сдвигаем 50 вправо: 5 9 40 50 [50]
Сдвигаем 40 вправо: 5 9 40 [40] [ ]
Сдвигаем 9 вправо: 5 9 [9] 40 [ ]
Сдвигаем 5 вправо: 5 [5] 9 40 [ ]
Вставляем 3 на позицию 0: [3] 5 9 40 50

Отсортированный массив: 3 5 9 40 50

```

2. Метод выбора

1. Постановка задачи

Отсортировать массив из n элементов по возрастанию.

2. Анализ задачи

1. Исходные данные: массив a размером n .
2. Массив делится на две логические части:
 - отсортированная — элементы $a[0 \dots i-1]$;
 - неотсортированная — элементы $a[i \dots n-1]$.

Изначально отсортированная часть пуста, граница находится перед первым элементом.

3. Внешний цикл $\text{for } (i = 0; i < n-1; i++)$ расширяет отсортированную часть:

а) Устанавливаем индекс минимального элемента: $\text{min} = i$ (предполагаем, что первый элемент неотсортированной части — минимальный).

б) Внутренний цикл $\text{for } (j = i+1; j < n; j++)$ перебирает оставшуюся неотсортированную часть:

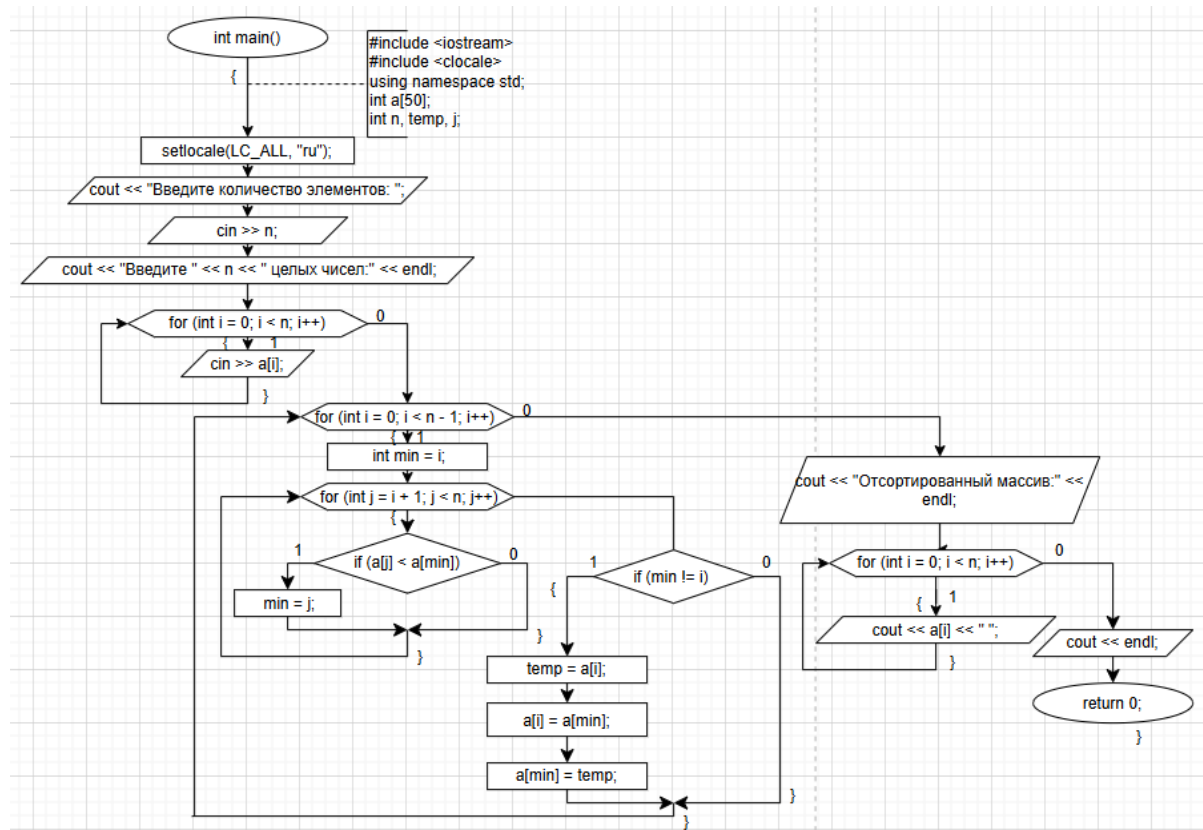
- если $a[j] < a[\text{min}]$, обновляем индекс: $\text{min} = j$.

в) После завершения внутреннего цикла меняем местами $a[i]$ и $a[\min]$ (если $\min \neq i$), тем самым помещая минимальный элемент в конец отсортированной части.

4. Граница отсортированной части автоматически сдвигается вправо на один элемент после каждой итерации внешнего цикла.

5. После $n-1$ итераций внешнего цикла массив полностью отсортирован по возрастанию (последний элемент автоматически занимает своё место).

3. Блок-схема



4. Код

```

#include <iostream>
#include <locale>
using namespace std;

int main() {
    setlocale(LC_ALL, "ru");
    int a[50];
    int n;

    cout << "Введите количество элементов: ";
    cin >> n;

    cout << "Введите " << n << " целых чисел:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    cout << "\nНачальный массив: ";
    for (int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }
    cout << "\n\n";

```

```

for (int i = 0; i < n - 1; i++) {
    int min = i;

    cout << "Шаг " << i + 1 << ": ищем минимальный элемент начиная с позиции "
<< i << endl;
    cout << "Текущее состояние: ";
    for (int k = 0; k < n; k++) {
        if (k == i) {
            cout << "[" << a[k] << " ";
        }
        else {
            cout << a[k] << " ";
        }
    }
    cout << endl;

    for (int j = i + 1; j < n; j++) {
        if (a[j] < a[min]) {
            min = j;
        }

        cout << " Сравниваем a[" << j << "]= " << a[j] << " с текущим минимумом
a[" << min << "]= " << a[min];
        if (j < n - 1) {
            if (a[j] < a[min]) {
                cout << " новый минимум";
            }
            else {
                cout << " минимум не изменился";
            }
        }
        cout << endl;
    }

    cout << " Найден минимальный элемент: " << a[min] << " на позиции " << min
<< endl;

    if (min != i) {
        cout << " Меняем местами a[" << i << "]= " << a[i] << " и a[" << min <<
"]= " << a[min] << endl;
        int temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }
    else {
        cout << " Элемент уже на своем месте" << endl;
    }

    cout << " Результат шага: ";
    for (int k = 0; k < n; k++) {
        if (k <= i) {
            cout << "[" << a[k] << " ";
        }
        else {
            cout << a[k] << " ";
        }
    }
    cout << "\n" << endl;
}

cout << "Отсортированный массив: ";
for (int i = 0; i < n; i++) {
    cout << a[i] << " ";
}

```

```

    }
    cout << endl;

    return 0;
}

```

5. Результат выполнения программы

```

Введите количество элементов: 5
Введите 5 целых чисел:
50
40
5
9
3

Начальный массив: 50 40 5 9 3

Шаг 1: ищем минимальный элемент начиная с позиции 0
Текущее состояние: [50] 40 5 9 3
Сравниваем a[1]=40 с текущим минимумом a[1]=40 минимум не изменился
Сравниваем a[2]=5 с текущим минимумом a[2]=5 минимум не изменился
Сравниваем a[3]=9 с текущим минимумом a[2]=5 минимум не изменился
Сравниваем a[4]=3 с текущим минимумом a[4]=3
Найден минимальный элемент: 3 на позиции 4
Меняем местами a[0]=50 и a[4]=3
Результат шага: [3] 40 5 9 50

Шаг 2: ищем минимальный элемент начиная с позиции 1
Текущее состояние: 3 [40] 5 9 50
Сравниваем a[2]=5 с текущим минимумом a[2]=5 минимум не изменился
Сравниваем a[3]=9 с текущим минимумом a[2]=5 минимум не изменился
Сравниваем a[4]=50 с текущим минимумом a[2]=5
Найден минимальный элемент: 5 на позиции 2
Меняем местами a[1]=40 и a[2]=5
Результат шага: [3] [5] 40 9 50

Шаг 3: ищем минимальный элемент начиная с позиции 2
Текущее состояние: 3 5 [40] 9 50
Сравниваем a[3]=9 с текущим минимумом a[3]=9 минимум не изменился
Сравниваем a[4]=50 с текущим минимумом a[3]=9
Найден минимальный элемент: 9 на позиции 3
Меняем местами a[2]=40 и a[3]=9
Результат шага: [3] [5] [9] 40 50

Шаг 4: ищем минимальный элемент начиная с позиции 3
Текущее состояние: 3 5 9 [40] 50
Сравниваем a[4]=50 с текущим минимумом a[3]=40
Найден минимальный элемент: 40 на позиции 3
Элемент уже на своем месте
Результат шага: [3] [5] [9] [40] 50

Отсортированный массив: 3 5 9 40 50

```

3. Метод обмена (Пузырька)

1. Постановка задачи

Отсортировать массив из n элементов по возрастанию.

2. Анализ задачи

- Исходные данные: массив a размером n .
- Массив логически делится на две части:
 - неотсортированная — элементы $a[0 \dots n-i-1]$;
 - отсортированная — элементы $a[n-i \dots n-1]$ (накапливается справа).
 Изначально отсортированная часть пуста.
- Внешний цикл $\text{for } (i = 0; i < n-1; i++)$ определяет количество проходов по массиву (максимум $n-1$ проходов):
 - Внутренний цикл $\text{for } (j = 0; j < n-i-1; j++)$ последовательно сравнивает соседние элементы в неотсортированной части:

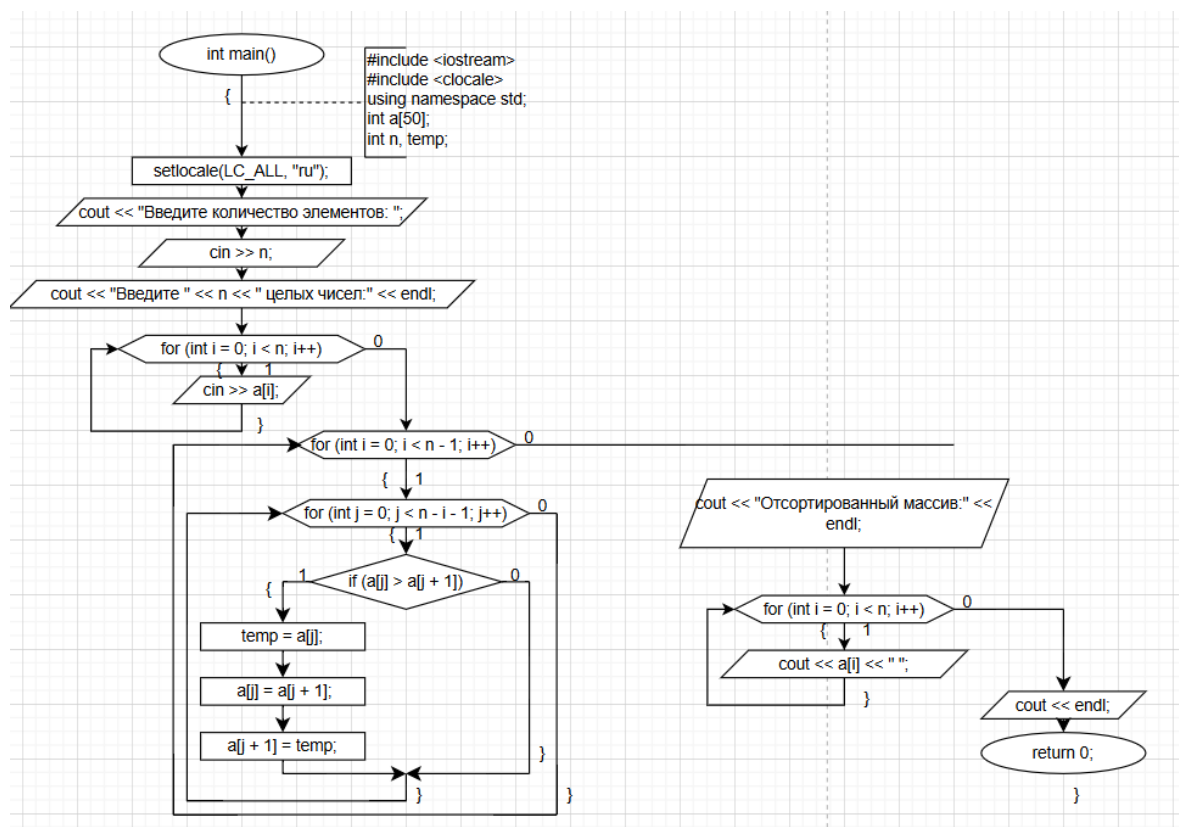
- если $a[j] > a[j+1]$, выполняем обмен: $temp = a[j]$; $a[j] = a[j+1]$; $a[j+1] = temp$.

б) После каждого прохода внутреннего цикла максимальный элемент неотсортированной части «всплывает» на правую границу и присоединяется к отсортированной части.

4. Граница отсортированной части сдвигается влево на один элемент после каждой итерации внешнего цикла (учтено в условии $j < n-i-1$).

5. После завершения внешнего цикла массив полностью отсортирован по возрастанию.

3. Блок-схема



4. Код

```
#include <iostream>
#include <locale>
using namespace std;

int main() {
    setlocale(LC_ALL, "ru");
    int a[50];
    int n;

    cout << "Введите количество элементов: ";
    cin >> n;

    cout << "Введите " << n << " целых чисел:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    cout << "Отсортированный массив:" << endl;
    for (int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }
    cout << endl;
    return 0;
}
```

```

cout << "\nНачальный массив: ";
for (int i = 0; i < n; i++) {
    cout << a[i] << " ";
}
cout << "\n\n";

int step = 1;
bool wasSwap;

for (int i = 0; i < n - 1; i++) {
    wasSwap = false;

    cout << "Проход " << step << ":" << endl;

    for (int j = 0; j < n - i - 1; j++) {
        cout << " Сравниваем пару (" << a[j] << ", " << a[j + 1] << "): ";
        for (int k = 0; k < n; k++) {
            if (k == j) {
                cout << "[" << a[k] << "];";
            }
            else if (k == j + 1) {
                cout << "[" << a[k] << " ]";
            }
            else {
                cout << a[k] << " ";
            }
        }

        if (a[j] > a[j + 1]) {
            cout << " - нужно поменять местами";

            int temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
            wasSwap = true;

            cout << "\n    Результат: ";
            for (int k = 0; k < n; k++) {
                if (k == j) {
                    cout << "[" << a[k] << "];";
                }
                else if (k == j + 1) {
                    cout << "[" << a[k] << " ]";
                }
                else {
                    cout << a[k] << " ";
                }
            }
            cout << endl;
        }
        else {
            cout << " - менять не нужно" << endl;
        }
    }

    cout << "    Результат прохода " << step << ":" << " ";
    for (int k = 0; k < n; k++) {
        if (k >= n - step) {
            cout << "[" << a[k] << " ]";
        }
        else {
            cout << a[k] << " ";
        }
    }
    cout << " (последние " << step << " элементов на своих местах)";
}

```

```

        if (!wasSwap) {
            cout << "\n  За проход не было обменов – сортировка завершена";
            break;
        }

        cout << "\n" << endl;
        step++;
    }

    cout << "\nОтсортированный массив: ";
    for (int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }
    cout << endl;

    return 0;
}

```

5. Результат выполнения программы

```

Введите количество элементов: 5
Введите 5 целых чисел:
40
50
9
5
3

Начальный массив: 40 50 9 5 3

Проход 1:
Сравниваем пару (40, 50): [40][50] 9 5 3 - менять не нужно
Сравниваем пару (50, 9): 40 [50][9] 5 3 - нужно поменять местами
Результат: 40 [9][50] 5 3
Сравниваем пару (50, 5): 40 9 [50][5] 3 - нужно поменять местами
Результат: 40 9 [5][50] 3
Сравниваем пару (50, 3): 40 9 5 [50][3] - нужно поменять местами
Результат: 40 9 5 [3][50]
Результат прохода 1: 40 9 5 3 [50] (последние 1 элементов на своих местах)

Проход 2:
Сравниваем пару (40, 9): [40][9] 5 3 50 - нужно поменять местами
Результат: [9][40] 5 3 50
Сравниваем пару (40, 5): 9 [40][5] 3 50 - нужно поменять местами
Результат: 9 [5][40] 3 50
Сравниваем пару (40, 3): 9 5 [40][3] 50 - нужно поменять местами
Результат: 9 5 [3][40] 50
Результат прохода 2: 9 5 3 [40] [50] (последние 2 элементов на своих местах)

Проход 3:
Сравниваем пару (9, 5): [9][5] 3 40 50 - нужно поменять местами
Результат: [5][9] 3 40 50
Сравниваем пару (9, 3): 5 [9][3] 40 50 - нужно поменять местами
Результат: 5 [3][9] 40 50
Результат прохода 3: 5 3 [9] [40] [50] (последние 3 элементов на своих местах)

Проход 4:
Сравниваем пару (5, 3): [5][3] 9 40 50 - нужно поменять местами
Результат: [3][5] 9 40 50
Результат прохода 4: 3 [5] [9] [40] [50] (последние 4 элементов на своих местах)

Отсортированный массив: 3 5 9 40 50

```